

HIBERNATE IN JAVA

Complete Interview Preparation Guide

Comprehensive Questions & Answers for Java Developers

Topics Covered

ORM Fundamentals • Architecture • Entity Mapping • HQL / Criteria API
Caching • Transactions • Fetch Strategies • Performance • Spring Integration

★ *Frequently asked questions are highlighted*

2024 – 2025 Edition | Java / Backend Developer Interviews

Table of Contents

Table of Contents.....	2
1. Hibernate Fundamentals.....	3
SessionFactory:.....	4
Session:.....	4
2. Hibernate Architecture.....	5
Key Components:.....	5
3. Entity Mapping & Annotations.....	7
@OneToOne.....	7
@OneToMany / @ManyToOne.....	8
@ManyToMany.....	8
4. Entity Lifecycle & States.....	9
5. Querying – HQL, JPQL & Criteria API.....	10
Solutions:.....	11
6. Hibernate Caching.....	12
First-Level Cache (L1).....	12
Second-Level Cache (L2).....	12
7. Transactions & Concurrency.....	14
Optimistic Locking.....	14
Pessimistic Locking.....	14
8. Fetch Strategies & Lazy Loading.....	16
Lazy Loading (FetchType.LAZY).....	16
Eager Loading (FetchType.EAGER).....	16
Prevention Strategies:.....	16
9. Inheritance Mapping.....	17
1. SINGLE_TABLE (Default).....	17
2. TABLE_PER_CLASS.....	17
3. JOINED.....	17
4. MAPPED_SUPERCLASS.....	17
10. Spring & Spring Boot Integration.....	18
Common Propagation Types:.....	18
11. Performance Best Practices.....	20
Fetching Optimization.....	20
Query Optimization.....	20
Batching.....	20
Caching.....	20
12. Advanced Topics.....	22
13. Quick Reference Cheat Sheet.....	24

1. Hibernate Fundamentals

★ FAQ Q: What is Hibernate and why is it used?

✔ Answer:

Hibernate is an open-source Object-Relational Mapping (ORM) framework for Java that maps Java classes to database tables and handles all CRUD operations transparently.

- Eliminates repetitive JDBC boilerplate (connection handling, PreparedStatement, ResultSet)
- Automatic DDL generation – tables can be created/updated automatically
- Database portability – switching databases requires minimal configuration changes
- Built-in caching (first-level and second-level) for improved performance
- HQL (Hibernate Query Language) enables database-agnostic queries
- Lazy loading reduces unnecessary data fetching

★ FAQ Q: What is ORM (Object-Relational Mapping)?

✔ Answer:

ORM is a programming technique that converts data between incompatible type systems (objects in OOP languages vs. relational tables in RDBMS). It bridges the 'impedance mismatch'.

- No SQL strings scattered through application code
- Type-safe data access using entity classes
- Automatic dirty checking – detects and persists modified objects
- Relationships modeled as Java object references

Q: What is the difference between Hibernate and JDBC?

✔ Answer:

JDBC is a low-level API requiring manual SQL writing, connection management, and result set parsing. Hibernate automates all of this on top of JDBC.

Feature	JDBC	Hibernate
SQL	Manually written	Auto-generated
Object mapping	Manual ResultSet parsing	Automatic via annotations
Caching	None	L1 + L2 built-in
Connection	Manual boilerplate	Session manages it
Portability	DB-specific SQL	HQL is DB-agnostic

★ FAQ Q: What are the core interfaces of Hibernate?

✔ Answer:

- SessionFactory: Thread-safe, heavyweight object created once per application. Responsible for creating Session objects.
- Session: Lightweight, non-thread-safe unit-of-work object. Used to interact with the database.
- Transaction: Abstracts JDBC/JTA transactions. Obtained from a Session.
- Query / NativeQuery: Represents HQL or native SQL queries.
- CriteriaBuilder: Type-safe programmatic query API (JPA standard).

- Configuration: Bootstraps Hibernate from hibernate.cfg.xml or programmatic setup.

★ **FAQ Q: What is the difference between SessionFactory and Session?**

✓ **Answer:**

SessionFactory is created once during application startup; Session is the main runtime interface for a single unit of work.

SessionFactory:

- Created once (expensive – reads config, sets up connection pool)
- Thread-safe – shared across the entire application
- Corresponds to one database; maintains second-level cache

Session:

- Created per request / unit of work (lightweight)
 - NOT thread-safe – must NOT be shared between threads
 - Wraps a JDBC Connection; maintains first-level (L1) cache automatically
-

2. Hibernate Architecture

★ FAQ Q: Explain the Hibernate architecture.

✓ Answer:

Hibernate sits between the Java application and the database, using several layers to translate object operations into database operations.

- Application Layer: Java code using entities and Hibernate APIs
- Hibernate Layer: SessionFactory, Session, Transaction, Query
- Backend API Layer: JDBC, JTA, JNDI used internally
- Database Layer: Actual RDBMS (MySQL, PostgreSQL, Oracle, etc.)

Key Components:

- Configuration: Reads hibernate.cfg.xml / application.properties and entity mappings
- SessionFactory: Thread-safe factory for Sessions; built from Configuration
- Session: Core interface; manages entity lifecycle; wraps JDBC connection
- Transaction: Demarcates unit of work; wraps JDBC/JTA transactions
- ConnectionProvider: Connection pool abstraction (c3p0, HikariCP, etc.)

Q: What is the Hibernate configuration file (hibernate.cfg.xml)?

✓ Answer:

hibernate.cfg.xml is the primary XML configuration file. It specifies database connection settings, dialect, entity mappings, and various behavioral options.

```
<?xml version="1.0" encoding="UTF-8"?>
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/mydb</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">secret</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <mapping class="com.example.Employee"/>
  </session-factory>
</hibernate-configuration>
```

Q: What are Hibernate dialects?

✓ Answer:

A Hibernate dialect is a class that tells Hibernate how to generate SQL for a particular database. Common dialects:

- MySQL8Dialect – for MySQL 8+
- PostgreSQLDialect – for PostgreSQL
- Oracle12cDialect – for Oracle 12c+
- SQLServerDialect – for MS SQL Server

- H2Dialect – for H2 (often used in testing)

❏ **Best Practice:** Always match the dialect exactly to your database version to ensure correct SQL generation.

★ **FAQ Q: What is hbm2ddl.auto and what are its options?**

✔ **Answer:**

The hibernate.hbm2ddl.auto property controls automatic DDL schema management at application startup.

- none: No DDL action; schema must exist manually (recommended for production)
- validate: Validates schema against entities; throws error if mismatched (good for production)
- update: Alters existing schema to match entities; never drops columns (dev/staging)
- create: Drops and recreates schema every startup (data is lost; use for testing)
- create-drop: Creates schema on startup and drops it on SessionFactory close (unit tests)

❏ **Best Practice:** Never use 'create' or 'create-drop' in production. Use 'validate' with Flyway or Liquibase for production schema management.

3. Entity Mapping & Annotations

★ FAQ Q: How do you map a Java class to a database table?

✓ Answer:

Using JPA/Hibernate annotations on the Java class. At minimum you need @Entity and @Id.

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false, length = 100)
    private String name;

    @Column(name = "email", unique = true)
    private String email;

    @Column(name = "salary", precision = 10, scale = 2)
    private BigDecimal salary;
    // getters and setters
}
```

★ FAQ Q: What are the different ID generation strategies?

✓ Answer:

- GenerationType.IDENTITY: Uses DB auto-increment (MySQL, PostgreSQL). Efficient but prevents batch inserts.
- GenerationType.SEQUENCE: Uses DB sequence object (Oracle, PostgreSQL). Allows batch inserts. Preferred for high-volume apps.
- GenerationType.TABLE: Simulates sequence using a special table. DB-agnostic but slow.
- GenerationType.AUTO: Hibernate chooses best strategy based on dialect. Default option.
- GenerationType.UUID: Generates UUID primary keys (JPA 3.1 / Hibernate 6+). Good for distributed systems.

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "emp_seq")
@SequenceGenerator(name = "emp_seq", sequenceName = "employee_sequence", allocationSize = 50)
private Long id;
```

★ FAQ Q: Explain the different types of relationships in Hibernate.

✓ Answer:

@OneToOne

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "address_id", referencedColumnName = "id")
private Address address;
```

@OneToMany / @ManyToOne

```
// Department side
@OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private List<Employee> employees = new ArrayList<>();

// Employee side
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "dept_id")
private Department department;
```

@ManyToMany

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "student_course",
    joinColumns = @JoinColumn(name = "student_id"),
    inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses = new HashSet<>();
```

★ FAQ Q: What is the difference between @JoinColumn and mappedBy?

✓ Answer:

- @JoinColumn: Placed on the owning side; specifies the foreign key column in the database table.
- mappedBy: Placed on the inverse (non-owning) side; tells Hibernate the relationship is already mapped by the specified field.

□ **Best Practice:** Only the owning side (@JoinColumn side) controls the foreign key. Changes made to the inverse side (mappedBy) are ignored during persistence.

Q: What cascade types does Hibernate support?

✓ Answer:

- CascadeType.PERSIST: Persisting parent also persists all children
- CascadeType.MERGE: Merging parent also merges all children
- CascadeType.REMOVE: Deleting parent also deletes all children
- CascadeType.REFRESH: Refreshing parent also refreshes children
- CascadeType.DETACH: Detaching parent also detaches children
- CascadeType.ALL: Applies all above cascades

□ **Best Practice:** Use CascadeType.REMOVE carefully – it can cause accidental mass deletions. Prefer orphanRemoval=true for @OneToMany.

4. Entity Lifecycle & States

★ FAQ Q: What are the different states of a Hibernate entity?

✓ Answer:

- Transient: Object created with 'new' but not associated with any Session. Not tracked by Hibernate.
- Persistent (Managed): Associated with an open Session. Changes auto-detected and synced to DB.
- Detached: Was persistent but its Session has been closed. Changes NOT tracked. Re-attach with `session.merge()`.
- Removed: Scheduled for deletion. Deleted from DB on commit.

```
new Employee()           → Transient
session.save(emp)        → Persistent  (INSERT on commit)
session.close()          → Detached
session.merge(emp)       → Persistent  (UPDATE on commit)
session.delete(emp)      → Removed     (DELETE on commit)
```

★ FAQ Q: What is dirty checking in Hibernate?

✓ Answer:

Dirty checking is Hibernate's automatic detection of changes to managed entities. At commit, Hibernate compares entity state with its snapshot and generates SQL UPDATE for changed fields only.

- No need to call `session.update()` for managed entities – changes are automatic
- Only columns that actually changed are updated
- Snapshot is stored in the first-level cache (Session cache)

★ FAQ Q: What is the difference between `get()` and `load()`?

✓ Answer:

- `session.get()`: Hits the database immediately. Returns null if not found. Returns actual entity object.
- `session.load()`: Returns a proxy object (lazy loading). Throws `ObjectNotFoundException` if proxy is accessed and entity missing.

📌 **Best Practice:** In JPA, equivalents are `EntityManager.find()` (like `get`) and `EntityManager.getReference()` (like `load`).

Q: What is the difference between `save()`, `saveOrUpdate()`, `persist()`, `merge()`, `update()`?

✓ Answer:

- `save()`: Returns generated identifier immediately; executes INSERT; Hibernate-proprietary.
- `persist()`: JPA standard; does not return identifier; INSERT happens at flush/commit.
- `update()`: Reattaches detached entity; always generates UPDATE.
- `merge()`: Copies detached entity state to new managed instance; safe regardless of state; JPA standard.
- `saveOrUpdate()`: Calls `save()` for transient, `update()` for detached; Hibernate-proprietary.

5. Querying – HQL, JPQL & Criteria API

★ FAQ Q: What is HQL (Hibernate Query Language)?

✓ Answer:

HQL is an object-oriented query language similar to SQL but operating on entities and their properties, not tables and columns. It is database-agnostic.

```
// Basic select
List<Employee> list = session.createQuery(
    "FROM Employee WHERE salary > :minSal", Employee.class)
    .setParameter("minSal", 50000)
    .getResultList();

// Join
List<Employee> list = session.createQuery(
    "SELECT e FROM Employee e JOIN e.department d WHERE d.name = :deptName",
    Employee.class)
    .setParameter("deptName", "Engineering")
    .getResultList();

// Aggregate
Long count = session.createQuery(
    "SELECT COUNT(e) FROM Employee e WHERE e.active = true", Long.class)
    .uniqueResult();
```

★ FAQ Q: What is the Criteria API and when is it preferred?

✓ Answer:

The Criteria API is a type-safe, programmatic way to build queries using Java objects, enabling compile-time checking and dynamic query building.

- Building dynamic queries with optional filter conditions
- IDE auto-completion and compile-time safety required
- Avoiding string concatenation for query building

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Employee> cq = cb.createQuery(Employee.class);
Root<Employee> root = cq.from(Employee.class);

Predicate salaryPred = cb.gt(root.get("salary"), 50000);
Predicate namePred = cb.like(root.get("name"), "%John%");
cq.where(cb.and(salaryPred, namePred));
cq.orderBy(cb.desc(root.get("salary")));

List<Employee> results = session.createQuery(cq).getResultList();
```

★ FAQ Q: What is the N+1 select problem and how do you solve it?

✓ Answer:

The N+1 problem occurs when loading N parent entities triggers N additional SELECT queries to load each entity's lazy-loaded collections.

```
// Problem: Each call to d.getEmployees() fires a SELECT
List<Department> depts = session.createQuery("FROM Department", Department.class).list();
for (Department d : depts) {
```

```
        System.out.println(d.getEmployees().size()); // N extra queries!
    }
```

Solutions:

- JOIN FETCH in HQL: "FROM Department d JOIN FETCH d.employees" – loads all in one query
- @BatchSize(size=10): Batch-loads collections – reduces N+1 to N/10+1
- @Fetch(FetchMode.SUBSELECT): Loads all collections in one extra query
- EntityGraph: Eagerly load specific associations per use case

```
// Best practice: JOIN FETCH
List<Department> depts = session.createQuery(
    "SELECT DISTINCT d FROM Department d JOIN FETCH d.employees", Department.class)
    .getResultList();
```

Q: What are Named Queries?

✓ Answer:

Named queries are pre-defined, reusable HQL or SQL queries declared at entity class level. They are parsed and validated at startup.

```
@Entity
@NamedQuery(name = "Employee.findByDept",
    query = "SELECT e FROM Employee e WHERE e.department.name = :deptName")
public class Employee { ... }

// Usage
List<Employee> emps = session.createNamedQuery("Employee.findByDept", Employee.class)
    .setParameter("deptName", "HR").getResultList();
```

6. Hibernate Caching

★ FAQ Q: Explain Hibernate's caching levels.

✓ Answer:

First-Level Cache (L1)

- Built into every Hibernate Session – always enabled, cannot be disabled
- Scoped to a single Session – not shared between Sessions
- Automatically stores entities loaded or saved during the Session
- Prevents duplicate queries for the same entity within the same Session

```
Employee e1 = session.get(Employee.class, 1L); // DB hit
Employee e2 = session.get(Employee.class, 1L); // Served from L1 cache - no DB hit
assert e1 == e2; // Same instance
```

Second-Level Cache (L2)

- Optional; shared across all Sessions of a SessionFactory
- Requires an external cache provider (Ehcache, Infinispan, Hazelcast, Redis)
- Configured per entity or collection using @Cache annotation

```
// application.properties
spring.jpa.properties.hibernate.cache.use_second_level_cache=true
spring.jpa.properties.hibernate.cache.region.factory_class=
    org.hibernate.cache.jcache.internal.JCacheRegionFactory

@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
public class Product { ... }
```

★ FAQ Q: What cache concurrency strategies are available?

✓ Answer:

- READ_ONLY: Best performance; only for data that never changes. No write support.
- NONSTRICT_READ_WRITE: Caches rarely written data; eventual consistency with short stale window.
- READ_WRITE: Ensures consistency using soft locks; suitable for occasionally updated data.
- TRANSACTIONAL: Full transactional cache (XA); requires JTA. Highest consistency, higher overhead.

Q: What is the Query Cache?

✓ Answer:

The Query Cache caches the results (IDs) of HQL/JPQL queries. When the same query runs again, Hibernate checks the cache before hitting the database.

- Must be enabled: `hibernate.cache.use_query_cache=true`
- Must also be enabled per query: `query.setCacheable(true)`
- Invalidated when any entity in the query result is modified

❏ **Best Practice:** Query Cache requires Second-Level Cache. Without L2 cache, query cache only stores IDs requiring individual DB lookups anyway.

7. Transactions & Concurrency

★ FAQ Q: How does Hibernate handle transactions?

✓ Answer:

Hibernate provides a Transaction interface wrapping the underlying JDBC transaction. The typical pattern: begin → operate → commit/rollback.

```
Session session = sessionFactory.openSession();
Transaction tx = null;
try {
    tx = session.beginTransaction();
    Employee emp = session.get(Employee.class, 1L);
    emp.setSalary(emp.getSalary().multiply(BigDecimal.valueOf(1.1)));
    tx.commit(); // dirty checking fires UPDATE automatically
} catch (Exception e) {
    if (tx != null) tx.rollback();
    throw e;
} finally {
    session.close();
}
```

□ **Best Practice:** With Spring's `@Transactional`, all of this boilerplate is handled automatically.

★ FAQ Q: What is optimistic vs. pessimistic locking in Hibernate?

✓ Answer:

Optimistic Locking

- Assumes conflicts are rare; does not lock rows in the database
- Uses a version column (`@Version`) to detect concurrent modifications
- Throws `OptimisticLockException` if another transaction modified the same row
- Best for high-read, low-write scenarios

```
@Entity
public class Account {
    @Id private Long id;
    @Version private int version; // Auto-managed by Hibernate
    private BigDecimal balance;
}
```

Pessimistic Locking

- Locks the database row immediately on read using `SELECT FOR UPDATE`
- Prevents other transactions from reading or modifying the locked row
- Use when conflicts are frequent or data integrity is critical

```
Employee emp = session.get(Employee.class, 1L, LockMode.PESSIMISTIC_WRITE);
```

Q: What is the difference between `session.flush()` and `session.clear()`?

✓ Answer:

- `flush()`: Synchronizes Session state to DB (executes pending SQL). Does NOT commit. Called

automatically before queries and at commit.

- **clear():** Evicts all entities from L1 cache. Useful for batch processing to prevent OutOfMemoryError. Does NOT execute pending SQL.
- **evict(entity):** Removes a specific entity from the L1 cache.

```
// Batch processing best practice
for (int i = 0; i < 100000; i++) {
    session.save(new Employee(...));
    if (i % 50 == 0) {
        session.flush(); // Write to DB
        session.clear(); // Free memory
    }
}
```

8. Fetch Strategies & Lazy Loading

★ FAQ Q: What is lazy loading vs. eager loading?

✓ Answer:

Lazy Loading (FetchType.LAZY)

- Association loaded only when first accessed in code
- Default for @OneToMany and @ManyToMany
- Reduces initial query cost but may cause LazyInitializationException if accessed after Session closes

Eager Loading (FetchType.EAGER)

- Association loaded immediately with the parent entity
- Default for @ManyToOne and @OneToOne
- Can cause performance issues – loads unnecessary data

```
@OneToMany(mappedBy = "department", fetch = FetchType.LAZY)
private List<Employee> employees;
```

★ FAQ Q: What is LazyInitializationException and how do you prevent it?

✓ Answer:

LazyInitializationException is thrown when a lazy-loaded collection or proxy is accessed after the Hibernate Session is closed.

Prevention Strategies:

- JOIN FETCH: Use HQL join fetch to eagerly load needed associations within the Session
- EntityGraph: Specify dynamic fetch plan without changing entity defaults
- DTO projection: Select only required data into DTOs, avoiding lazy proxies entirely
- Hibernate.initialize(): Explicitly initialize proxy before Session closes
- Open Session in View: Keeps Session open during view rendering (anti-pattern for APIs)

```
@NamedEntityGraph(name = "Employee.withDept",
    attributeNodes = @NamedAttributeNode("department"))
@Entity
public class Employee { ... }

// Usage
EntityGraph graph = em.getEntityGraph("Employee.withDept");
Map hints = Map.of("javax.persistence.fetchgraph", graph);
Employee emp = em.find(Employee.class, 1L, hints);
```


9. Inheritance Mapping

★ **FAQ Q: What inheritance mapping strategies does Hibernate support?**

✓ **Answer:**

1. SINGLE_TABLE (Default)

- All classes in hierarchy stored in one table with a discriminator column
- Pro: Best performance (single join); Con: Many nullable columns

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "emp_type", discriminatorType = DiscriminatorType.STRING)
public abstract class Person { @Id Long id; String name; }

@Entity @DiscriminatorValue("EMP")
public class Employee extends Person { String department; }
```

2. TABLE_PER_CLASS

- Each concrete class has its own table with ALL fields (including inherited)
- Pro: No nullable columns; Con: UNION queries needed for polymorphic queries

```
@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)
```

3. JOINED

- Base class has its own table; subclass table has only extra fields + FK to parent
- Pro: Normalized schema, no nulls; Con: JOIN required for each query

```
@Inheritance(strategy = InheritanceType.JOINED)
```

4. MAPPED_SUPERCLASS

- @MappedSuperclass is not an entity – not stored in its own table
- Provides shared field mappings to subclasses; no polymorphic queries possible

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id @GeneratedValue Long id;
    @CreationTimestamp LocalDateTime createdAt;
}
```

10. Spring & Spring Boot Integration

★ FAQ Q: How is Hibernate integrated with Spring Boot?

✓ Answer:

Spring Boot auto-configures Hibernate as the JPA provider when spring-boot-starter-data-jpa is on the classpath.

```
<!-- Maven dependency -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

★ FAQ Q: What is Spring Data JPA?

✓ Answer:

Spring Data JPA provides a repository abstraction on top of JPA/Hibernate, eliminating boilerplate data access code.

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    // Method name query derivation
    List<Employee> findByDepartmentNameAndSalaryGreaterThan(String dept, BigDecimal
salary);

    // Custom JPQL query
    @Query("SELECT e FROM Employee e WHERE e.salary = (SELECT MAX(e2.salary) FROM
Employee e2)")
    Employee findHighestPaid();

    // Native SQL
    @Query(value = "SELECT * FROM employees WHERE YEAR(hire_date) = :year", nativeQuery =
true)
    List<Employee> findHiredInYear(@Param("year") int year);
}
```

★ FAQ Q: How does @Transactional work with Hibernate in Spring?

✓ Answer:

Spring's @Transactional annotation wraps method execution in a transaction, managing begin/commit/rollback automatically using AOP proxies.

- Applied at class or method level on @Service classes
- readOnly = true optimizes read-only transactions (no flush, DB optimizations)
- rollbackFor specifies exception types that trigger rollback

Common Propagation Types:

- REQUIRED (default): Join existing transaction; create new if none

- **REQUIRES_NEW**: Always create a new transaction; suspend current if any
- **SUPPORTS**: Join if exists; run non-transactionally if none
- **NOT_SUPPORTED**: Always run non-transactionally; suspend current if any
- **NEVER**: Must run non-transactionally; throw exception if transaction exists

```
@Service
@Transactional
public class EmployeeService {
    @Transactional(readOnly = true)
    public List<Employee> getAllEmployees() {
        return employeeRepository.findAll();
    }

    @Transactional(rollbackFor = BusinessException.class)
    public void processPayroll(Long empId) { ... }
}
```

11. Performance Best Practices

★ FAQ Q: What are the key performance best practices in Hibernate?

✓ Answer:

Fetching Optimization

- Always use FetchType.LAZY for collections (@OneToMany, @ManyToMany)
- Use JOIN FETCH for use-case-specific eager loading rather than global EAGER
- Use DTO projections (SELECT new com.example.EmployeeDTO(...)) to avoid loading full entities
- Prefer EntityGraph over fetch joins when building reusable fetch plans

Query Optimization

- Use pagination (setFirstResult / setMaxResults) to avoid loading large result sets
- Use named queries and Criteria API for type-safe, compiled queries
- Avoid SELECT * in native queries – select only required columns

Batching

- Enable JDBC batching: hibernate.jdbc.batch_size=30
- Use StatelessSession for bulk import/export – bypasses session cache entirely
- Flush and clear in loops to prevent OutOfMemoryError during bulk operations

Caching

- Enable L2 cache for reference/configuration data (countries, categories, etc.)
- Use READ_ONLY strategy for immutable data
- Avoid caching frequently updated entities

Q: What is @DynamicInsert and @DynamicUpdate?

✓ Answer:

These annotations optimize generated SQL by including only non-null (insert) or changed (update) columns rather than all columns.

```
@Entity
@DynamicInsert // Only include non-null fields in INSERT
@DynamicUpdate // Only include changed fields in UPDATE
public class Employee { ... }
```

❏ **Best Practice:** @DynamicUpdate disables prepared statement caching, so benchmark before using in high-throughput scenarios.

★ FAQ Q: How do you implement pagination in Hibernate?

✓ Answer:

```
int pageNumber = 2;
int pageSize = 20;
List<Employee> page = session.createQuery("FROM Employee ORDER BY name", Employee.class)
```

```
.setFirstResult((pageNumber - 1) * pageSize) // offset
.setMaxResults(pageSize)                    // limit
.getResultList();

// With Spring Data JPA
Page<Employee> page = employeeRepository.findAll(PageRequest.of(1, 20, Sort.by("name")));
```

12. Advanced Topics

★ FAQ Q: What is the difference between Hibernate and JPA?

✔ Answer:

- JPA (Jakarta Persistence API) is a specification – a set of interfaces and rules defined in Java EE / Jakarta EE.
- Hibernate is the most popular implementation of the JPA specification.
- JPA defines the API: @Entity, @Id, EntityManager, JPQL, etc.
- Hibernate implements JPA and adds its own extensions (HQL, @Cache, etc.)
- Other JPA implementations: EclipseLink, OpenJPA, DataNucleus

📌 **Best Practice:** Code to the JPA standard; use Hibernate extensions only when needed to keep code portable.

Q: What is a StatelessSession and when would you use it?

✔ Answer:

StatelessSession is a stripped-down session with no first-level cache, no dirty checking, and no cascading. Designed for bulk data operations.

- No first-level cache – no risk of OutOfMemoryError in batch processing
- No dirty checking – must call explicit update()
- No cascading – associations must be handled manually
- Better performance for high-volume INSERT/UPDATE/DELETE

```
StatelessSession ss = sessionFactory.openStatelessSession();
Transaction tx = ss.beginTransaction();
for (Employee emp : largeDataList) {
    ss.insert(emp);
}
tx.commit();
ss.close();
```

Q: What are Hibernate interceptors and event listeners?

✔ Answer:

- Interceptors: Implement org.hibernate.Interceptor interface. Methods: onSave(), onFlushDirty(), onLoad(), onDelete().
- Event Listeners: Implement specific interfaces (PreInsertEventListener, PostUpdateEventListener, etc.). More granular than interceptors.

```
// JPA lifecycle callbacks
@Entity
public class Employee {
    @PrePersist
    public void onPrePersist() { this.createdAt = LocalDateTime.now(); }

    @PreUpdate
    public void onPreUpdate() { this.updatedAt = LocalDateTime.now(); }
}
```

Q: What is @Embeddable and @Embedded?

✓ Answer:

@Embeddable defines a class whose state is mapped as part of the owning entity's table (no separate table).
 @Embedded embeds it.

```
@Embeddable
public class Address {
    private String street;
    private String city;
    private String country;
}

@Entity
public class Employee {
    @Id Long id;
    @Embedded
    @AttributeOverrides({
        @AttributeOverride(name = "street", column = @Column(name = "home_street"))
    })
    private Address homeAddress;
}
```

Q: How does Hibernate handle enums?

✓ Answer:

- EnumType.ORDINAL (default): Stores integer ordinal (0, 1, 2...). Fragile – avoid in production.
- EnumType.STRING: Stores the enum name as a String. More readable and safe.

```
@Column(name = "status")
@Enumerated(EnumType.STRING)
private EmployeeStatus status; // Stored as "ACTIVE", "INACTIVE", etc.
```

❏ **Best Practice:** Always prefer EnumType.STRING in production to avoid data corruption when enum constants are added or reordered.

13. Quick Reference Cheat Sheet

Core Annotations

Annotation	Purpose
@Entity	Marks class as a persistent entity
@Table(name)	Specifies database table name
@Id	Marks primary key field
@GeneratedValue	Configures primary key generation strategy
@Column	Customizes column name, length, nullable, unique
@Transient	Excludes field from persistence
@Lob	Maps large objects (CLOB, BLOB)
@Enumerated	Maps enum fields (use EnumType.STRING)
@Version	Enables optimistic locking with version column
@Embeddable / @Embedded	Embeds value object into owning entity's table

Relationship Annotations

Annotation	Purpose
@OneToOne	One-to-one association
@OneToMany	One-to-many (use FetchType.LAZY)
@ManyToOne	Many-to-one (owning side)
@ManyToMany	Many-to-many (requires @JoinTable)
@JoinColumn	Specifies foreign key column (owning side)
@JoinTable	Specifies join table for @ManyToMany
mappedBy	Non-owning side of bidirectional relationship

Session Methods Quick Reference

Method	Behavior
session.get(Class, id)	Hits DB immediately; returns null if not found
session.load(Class, id)	Returns proxy; throws exception if not found on access
session.save(entity)	INSERT; returns generated ID; Hibernate-only
session.persist(entity)	INSERT at flush; JPA standard; no return value
session.merge(entity)	Copies state to managed instance; JPA standard
session.update(entity)	Reattaches detached entity; generates UPDATE
session.delete(entity)	Marks entity as Removed; DELETE on commit
session.flush()	Executes pending SQL; does NOT commit
session.clear()	Evicts all entities from L1 cache
session.evict(entity)	Removes specific entity from L1 cache

hbm2ddl.auto Options

Option	Behavior	Environment
none	No DDL action	Production ✓
validate	Validates schema only	Production ✓
update	Alters schema (no drops)	Dev / Staging
create	Drops & recreates schema	Testing
create-drop	Creates & drops on close	Unit Tests

Cache Concurrency Strategies

Strategy	Use Case
READ_ONLY	Immutable reference data (country list, config)
NONSTRICT_READ_WRITE	Rarely updated data; small stale window acceptable
READ_WRITE	Occasionally updated data; requires consistency
TRANSACTIONAL	Strict consistency; requires JTA; highest overhead

Inheritance Strategies

Strategy	Pro	Con
SINGLE_TABLE	Best performance, single join	Many nullable columns
TABLE_PER_CLASS	No nullable columns	UNION queries; no shared columns
JOINED	Normalized schema; no nulls	JOIN required for every query
MAPPED_SUPERCLASS	Shared mappings without entity table	No polymorphic queries possible

★ = Frequently Asked Interview Questions

Good luck with your interview! Practice the code examples and understand the concepts thoroughly.